Контест 13 · Надёжность и время

Неделя 26 · темы недель 24–26: длительности и часовые пояса, расписания, повторы и идемпотентность, доступность и инциденты, время в распределённой системе; в проектной части — UX, бэкапы и runbook. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

> Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

A. Время

A1. `parse_duration(s)` — `"1h30m"`, `"2d"`, `"45s"`, `"1h 5m"` → секунды. Единицы `d`, `h`, `m`, `s` идут по убыванию, каждая не больше раза, числа целые, пробелы игнорируются. Иначе (и пустая строка) — `ValueError`.

```
Ввод:  parse_duration("1h30m"), parse_duration("2d"), parse_duration("1h 5m")
       parse_duration("30m1h")
Вывод: 5400, 172800, 3900
       ValueError
```

A2. `utc_to_local(ts, tz)` — момент UTC `"ГГГГ-ММ-ДДТЧЧ:ММ:ССZ"` → местное время зоны `tz` в ISO со смещением (`zoneinfo`). `local_day_bounds(day, tz)` — начало и конец местных суток в UTC. Почему сутки 28.03.2027 в Берлине короче 24 часов?

```
Ввод:  utc_to_local("2027-03-28T01:30:00Z", "Europe/Berlin")
       utc_to_local("2027-04-26T21:30:00Z", "Europe/Moscow")
       local_day_bounds("2027-03-28", "Europe/Berlin")
Вывод: "2027-03-28T03:30:00+02:00"
       "2027-04-27T00:30:00+03:00"
       ("2027-03-27T23:00:00Z", "2027-03-28T22:00:00Z")
```

A3. `uptime(outages, start, end)` — доступность за `[start, end)` в процентах, 3 знака. Простои `(начало, конец)` могут пересекаться и выходить за границы периода.

```
Ввод:  uptime([(100, 200), (150, 300), (900, 1100)], 0, 1000), uptime([], 0, 10)
Вывод: 70.0, 100.0
```

A4*. `cron_next(expr, after)` — упрощённый cron `"минута час * * день_недели"`: в полях `*`, число, список через запятую, диапазон `a-b`, шаг `*/n`; день недели 0–6, 0 — воскресенье. Ближайший момент строго после `after`. День месяца и месяц не `*` — `ValueError`.

```
Ввод:  cron_next("0 3 * * *", datetime(2027, 4, 26, 3, 0))
       cron_next("*/15 9-18 * * 1-5", datetime(2027, 4, 30, 18, 50))
Вывод: datetime(2027, 4, 27, 3, 0)
       datetime(2027, 5, 3, 9, 0)
```

B. Повторы

B1. `backoff_schedule(n, base, cap, seed=None)` — паузы перед повторами 1…n: `min(cap, base · 2^(i−1))`. С `seed` — «полный джиттер»: `rng = random.Random(seed)`, пауза `rng.uniform(0, та же величина)`. 3 знака. Зачем джиттер, если клиентов тысяча?

```
Ввод:  backoff_schedule(6, 0.5, 5), backoff_schedule(4, 1, 10, seed=58)
Вывод: [0.5, 1.0, 2.0, 4.0, 5.0, 5.0], [0.581, 0.411, 2.568, 7.572]
```

B2. `retry(fn, attempts, retry_on=(Exception,), base=0.1, cap=5.0, sleep=None)` — вызвать `fn()` до `attempts` раз. Исключение из `retry_on` — `sleep(пауза из B1)` и снова; другое — пробросить сразу. Успех — `(результат, номер попытки)`. Попытки кончились — пробросить последнее исключение.

```
Ввод:  sl = []; retry(flaky, 5, (ConnectionError,), sleep=sl.append); sl
Вывод: ("ok", 3); [0.1, 0.2]          # flaky падает дважды, потом "ok"
```

B3. `IdempotencyStore(ttl)` — ключ идемпотентности. `handle(key, body, fn, now=0)`: первый раз — `fn(body)` и запомнить ответ; тот же `key` и `body` — тот же ответ без вызова; тот же `key` с другим `body` — `Conflict`. Упавший `fn` не запоминаем. Ключ живёт `ttl` секунд.

```
Ввод:  s = IdempotencyStore(100)
       s.handle("k1", {"x": 1}, fn), s.handle("k1", {"x": 1}, fn, 50)
       s.handle("k1", {"x": 2}, fn, 60)
Вывод: {"order": 1}, {"order": 1}
       Conflict
```

C. Надёжность

C1. `availability(spec)` — доступность системы: число — компонент; `("series", [...])` — нужны все; `("parallel", [...])` — хватит одного. 6 знаков. Что выгоднее: вторая реплика API или второй Redis?

```
Ввод:  availability(("series", [0.999, ("parallel", [0.99, 0.99]), 0.9995]))
       availability(("parallel", [0.9, 0.9, 0.9]))
Вывод: 0.998401
       0.999
```

C2. `downtime_budget(slo_percent, days=30)` — допустимый простой в минутах, 1 знак.

```
Ввод:  downtime_budget(99.9), downtime_budget(99.5), downtime_budget(99.99, 365)
Вывод: 43.2, 216.0, 52.6
```

C3. `incident_stats(outages, start, end)` — простои в секундах, пересекающиеся — один инцидент. `{"count", "mttr_min" — средняя длительность в минутах, "mtbf_h" — (период − простой) / count в часах}`, 1 знак. Нет инцидентов — `mttr_min` 0.0, `mtbf_h` `None`.

```
Ввод:  incident_stats([(3600, 4200), (4000, 4800), (86400, 87000)], 0, 7 * 86400)
Вывод: {"count": 2, "mttr_min": 15.0, "mtbf_h": 83.8}
```

C4*. `health_transitions(checks, fail_after=3, recover_after=2)` — проверки здоровья (`True` — ответил). Сначала `"up"`; в `"down"` — после `fail_after` неудач подряд, обратно — после `recover_after` удач подряд. Ответ — `(индекс проверки, новое состояние)`. Зачем такой гистерезис?

```
Ввод:  health_transitions([True, False, False, True, False, False, False, False,
                           True, False, True, True, True])
Вывод: [(6, "down"), (11, "up")]
```

D. Время в распределённой системе

D1. `lamport(events)` — часы Лэмпорта. Событие `(процесс, "local")`, `(процесс, "send", id)` или `(процесс, "recv", id)`. `local` и `send`: часы `+1`; `recv`: `max(часы, метка сообщения) + 1`. Ответ — метка каждого события.

```
Ввод:  lamport([("A", "local"), ("A", "send", "m1"), ("B", "local"), ("B", "local"),
                ("B", "local"), ("B", "recv", "m1"), ("B", "send", "m2"),
                ("A", "recv", "m2")])
Вывод: [1, 2, 1, 2, 3, 4, 5, 6]
```

D2. `vc_compare(a, b)` — векторные часы (нет ключа — 0): `"equal"`, `"before"`, `"after"` или `"concurrent"`.

```
Ввод:  vc_compare({"A": 1, "B": 2}, {"A": 1, "B": 3}), vc_compare({"A": 2}, {"B": 1})
       vc_compare({"A": 1}, {"A": 1, "B": 0})
Вывод: "before", "concurrent"
       "equal"
```

D3. `lww_merge(*replicas)` — регистр «последняя запись побеждает»: ключ → `(время, узел, значение)`. Побеждает большее время, при равенстве — больший узел. Ответ — ключ → значение по алфавиту ключей. Почему результат не зависит от порядка реплик?

```
Ввод:  lww_merge({"skin": (5, "api1", "red"), "lang": (1, "api1", "ru")},
                 {"skin": (5, "api2", "blue"), "sound": (2, "api2", "off")})
Вывод: {"lang": "ru", "skin": "blue", "sound": "off"}
```

E. Проект и алгоритмы

E1. Отчёт UX (семинар 25): три проблемы с тяжестью и долей участников, одна исправленная P1 и повторный тест после правки.

E2. Runbook восстановления (лекция 26) проходит проверку из лабораторной 20 и восстановлен «вслепую» на семинаре: покажите время восстановления и сверку строк.

E3. Ночной бэкап базы staging: cron или сервис, ротация, копия не на том же диске. Покажите вчерашний файл и его sha256.

E4 · босс. Очередь задач с повторами (награды, рассылки бота). За каждую часть свой плюс.

а) `next_attempt_at(fail_time, attempt, base=1, cap=60)` — после неудачной попытки `attempt` (с 1) следующая через `min(cap, base · 2^(attempt − 1))`.

```
Ввод:  next_attempt_at(10, 1), next_attempt_at(10, 4)
Вывод: 11.0, 18.0
```

б) `simulate(jobs, max_attempts=4, base=1, cap=60)` — задача: `id → (время постановки, исходы попыток)`; список кончился — дальше неудачи. Попытки мгновенны. Ответ: `id → ("done" | "dead", попыток, время последней попытки)`. `timeline(jobs)` — все попытки по времени: `(время, id, попытка)`, при равенстве — по `id`.

```
Ввод:  J = {"a": (0, [False, False, True]), "b": (1, [True]), "c": (2, [])}; simulate(J)
Вывод: {"a": ("done", 3, 3.0), "b": ("done", 1, 1), "c": ("dead", 4, 9.0)}
```

в) `apply_rewards(deliveries)` — доставка «хотя бы один раз»: `(id события, игрок, монеты)` может прийти повторно. Начислить каждое событие один раз; тот же `id` с другим содержимым — `ValueError`. Ответ — баланс по алфавиту игроков.

```
Ввод:  apply_rewards([("e1", "ann", 10), ("e2", "bob", 5), ("e1", "ann", 10),
                      ("e3", "ann", 3)])
Вывод: {"ann": 13, "bob": 5}
```

E5*. Аренда лидерства с fencing-токенами: только один экземпляр бота рассылает награды. `Lease(ttl).acquire(owner, now)` — свободна или истекла — новый токен (на 1 больше прошлого), занята другим — `None`, тем же — продлить. `Storage.write(token, value)` — отказ, если токен меньше уже виденного. `run(script, ttl=10)` — `("acq", кто, время)`, `("write", токен, значение)`, `("rel", кто, токен)` → результаты по порядку. Почему без токенов «зомби» А испортил бы данные?

```
Ввод:  run([("acq", "A", 0), ("write", 1, "a1"), ("acq", "B", 5), ("acq", "B", 12),
            ("write", 2, "b1"), ("write", 1, "a2"), ("acq", "A", 13), ("rel", "B", 2),
            ("acq", "A", 14), ("write", 3, "a3")])
Вывод: [1, True, None, 2, True, False, None, True, 3, True]
```